Python for LeetCode

Python from zero, up to what you need for your first 50 easy problems: numbers, if, functions, strings, lists, loops, dictionaries, sets and tuples.

How to use this: read one lesson, run its example script, then solve that lesson's stage in the **practice list** (the other PDF). Then the next lesson.

Lessons

1 Run Python

2 Variables and numbers

3 If and else

4 Functions

5 How a LeetCode problem looks

6 Strings

7 Lists

8 Loops

9 Dictionaries

10 Sets and tuples

LESSON 1

Run Python

After this page you'll be able to run Python, print things on the screen, and write and run your own `.py` file.

A program is a recipe

A recipe is a list of steps: boil water, add tea, add milk, wait two minutes. You follow the steps from top to bottom, one at a time.

A **program** is the same thing for a computer. It's a list of steps, written down, that the computer follows from top to bottom. **Python** is the language we write those steps in. It looks a bit like English, which makes it a good first language.

The computer is a very careful cook, though. It does exactly what the recipe says, nothing more. If a word is misspelled, it stops and tells you. That's normal, and you'll see it happen on this page.

Check that Python is there

Open a **terminal** (the window where you type commands) and type:

```
python3 --version
```

You'll see something like:

```
Python 3.12.3
```

The exact number may be a bit different. Anything starting with `Python 3` is fine.

Talk to Python directly

Type `python3` on its own and press Enter:

```
python3
```

```
Python 3.12.3 (main, Aug 31 2026, 10:18:26) [GCC 13.3.0] on linux
Type "help", "copyright", "credits" or "license" for more information.
>>>
```

The `>>>` is called the **prompt**. It means "Python is listening, type a line". Whatever you type, Python runs it straight away and shows the answer. This is called the **interactive prompt**, and it's a good place to try small things.

Type these, one line at a time:

```
>>> print("Hello")
Hello
>>> print(2 + 3)
5
>>> print("2 + 3")
2 + 3
```

- `print(...)` shows whatever is inside the brackets on the screen.
- Text goes inside **quotes** (`"Hello"`). Python shows it exactly as written.
- Without quotes, `2 + 3` is maths, so Python works out the answer first and prints `5`.
- With quotes, `"2 + 3"` is only text, so it prints `2 + 3`.

To leave, type:

```
>>> exit()
```

You're back in the normal terminal.

Write a program in a file

The interactive prompt forgets everything when you close it. To keep a program, write it in a **file**. Python files end in `.py`.

Make a new file called `hello.py` in your editor and put this one line in it:

```
print("Hello, Tanvi!")
```

Save it. In the terminal, go to the folder where you saved it and run:

```
python3 hello.py
```

```
Hello, Tanvi!
```

That's your first program. `python3 hello.py` means "Python, read this file and follow the steps in it".

Many steps, top to bottom

Change `hello.py` to this:

```
print("Hello, Tanvi!")
print("I am learning Python.")
print("I have", 3, "cats")
print()
print("The end.")
```

Run it again:

```
python3 hello.py
```

```
Hello, Tanvi!
I am learning Python.
I have 3 cats

The end.
```

- Python ran the lines in order, top to bottom, like a recipe.
- You can give `print` several things, separated by **commas**. It puts a space between them.
- `print()` with nothing inside prints an empty line.

Comments: notes for people

A line that starts with `#` is a **comment**. Python skips it completely. Comments are notes for whoever reads the code (including you, next week).

```
# This line is a comment. Python ignores it.
print("This line runs.")   # a comment can also sit at the end of a line
```

```
This line runs.
```

Errors are normal

Everyone who writes code sees errors every day. An error isn't you breaking something. It's Python saying "I didn't understand this line, here's where and why".

Make a small typo on purpose. Put this in `hello.py` (notice `prnt`, with a missing `i`):

```
print("Hello")
prnt("Bye")
```

Run it:

```
Hello
Traceback (most recent call last):
  File "/home/tanvi/hello.py", line 2, in <module>
    prnt("Bye")
    ^^^^
NameError: name 'prnt' is not defined. Did you mean: 'print'?
```

(The path after `File` will be wherever your file is saved.)

That block of text is called a **traceback**. It looks scary, but read it like this:

1. **Read the last line first.** It says what went wrong: `NameError: name 'prnt' is not defined`. Python doesn't know any word called `prnt`. It even guesses what you meant: `Did you mean: 'print'?`
2. **Then look up for the line number.** `line 2` tells you where. The `^^^^` points at the exact word.
3. Notice that `Hello` was printed first. Python followed the recipe until it hit the step it didn't understand, then stopped.

Fix the typo, run it again, and it works.

Try it

From the `learn` folder, run `examples/01-run-python.py`:

```
python3 examples/01-run-python.py
```

```
Hello, Tanvi!
This is my first Python program.
7
5
2 + 3
5
I have 3 cats

The end.
```

Then open the file and change things: print your own name, print `12 * 12` (`*` means "times"), and remove one letter from a `print` to see the error.

Common mistakes

Forgetting a closing bracket or quote. If you write `print("one"` with no `)` at the end, you'll see:

```
SyntaxError: '(' was never closed
```

A **SyntaxError** means Python couldn't even read the line, because something is missing or out of place. Here, a `(` was opened and never closed. Count your brackets and quotes: every `(` needs a `)`, every `"` needs another `"`.

Typing `python3 hello.py` inside the `>>>` prompt. That's a terminal command, not Python. If you see `>>>`, type `exit()` first, then run the file.

Check yourself

1. What's the difference between `print(4 + 4)` and `print("4 + 4")`?
2. In a traceback, which line do you read first?
3. What does a line starting with `#` do?

Next

Next you'll store values in named boxes and do maths with them: Variables and numbers.

Example script: `examples/01-run-python.py`

Run it from the `learn` folder, then change things and run it again.

```python
# Page 1: Run Python
# Run me with:  python3 01-run-python.py
#
# Lines that start with # are comments. Python skips them.
# They're notes for people reading the code.

print("Hello, Tanvi!")
print("This is my first Python program.")

# print can show numbers too. Python works out the maths first.
print(7)
print(2 + 3)

# Words go inside quotes. Numbers don't.
print("2 + 3")      # quotes: prints the text exactly as written
print(2 + 3)        # no quotes: prints the answer

# Give print several things, separated by commas.
# It puts a space between them.
print("I have", 3, "cats")

# print() with nothing inside prints an empty line.
print()
print("The end.")

# Try changing things:
#   1. Print your own name.
#   2. Print the answer to 12 * 12  (* means "times").
#   3. Remove one letter from a print, run it, and read the error's last line.
```

LESSON 2

Variables and numbers

After this page you'll be able to store values under a name and do maths with them, including the two operators LeetCode loves: `//` and `%`.

A labelled box

Think of a box with a label stuck on it. The label says `age`, and inside the box is `20`. Later you can ask "what's in the `age` box?" and get `20` back. You can also take the old thing out and put something new in.

In Python, that box is called a **variable**. You make one with `=`:

```
age = 20
print(age)
```

```
20
```

- `age` is the **name** (the label).
- `20` is the **value** (what's inside).
- `=` means "put this value into this box". It is not "equals" like in maths. Read `age = 20` as "age gets 20".

Names can use letters, numbers and `_`, but can't start with a number and can't have spaces. `my_age` is fine. `my age` and `2age` are not.

Kinds of values

Values come in different kinds. Python calls the kind of a value its **type**. Four you'll use all the time:

```
age = 20             # int: a whole number
price = 49.5         # float: a number with a decimal point
name = "Tanvi"       # str: text, always inside quotes
is_raining = False   # bool: True or False, nothing else
```

- **int** (short for integer) is a whole number: `20`, `0`, `-7`.
- **float** is a number with a decimal point: `49.5`, `3.0`.
- **str** (short for string) is text: `"Tanvi"`, `"hello"`. You'll learn much more about strings on page 6.
- **bool** (short for boolean) is either `True` or `False`. Capital T and F. You'll use these a lot on the next page.

`type()` tells you which kind a value is:

```
print(type(age))
print(type(price))
print(type(name))
print(type(is_raining))
```

```
<class 'int'>
<class 'float'>
<class 'str'>
<class 'bool'>
```

Maths

```
print(7 + 2)    # add
print(7 - 2)    # subtract
print(7 * 2)    # multiply
print(7 / 2)    # divide
print(7 // 2)   # divide, keep only the whole part
print(7 % 2)    # remainder: what's left over
print(7 ** 2)   # power: 7 times 7
```

```
9
5
14
3.5
3
1
49
```

Brackets work like in school maths: `(2 + 3) * 4` is `20`, while `2 + 3 * 4` is `14`.

`/` always gives a float

```
print(10 / 2)
```

```
5.0
```

Even when it divides evenly, `/` gives `5.0`, not `5`. That `.0` matters on LeetCode: if a problem wants a whole number back, use `//`, not `/`.

`//` and `%`: sharing sweets

You have 7 sweets and 2 friends. Each friend gets 3 (that's `7 // 2`), and 1 sweet is left over (that's `7 % 2`).

```
  7 sweets, 2 friends

  friend 1:  ● ● ●
  friend 2:  ● ● ●
  left over: ●

  7 // 2  ->  3   (how many each)
  7 %  2  ->  1   (how many left)
```

- `//` is **floor division**: how many whole times it fits.
- `%` is **modulo** (say "mod"): what's left over.

They show up everywhere. Some everyday uses:

Minutes into hours and minutes. A film is 135 minutes long:

```
minutes = 135
print(minutes // 60, "hours and", minutes % 60, "minutes")
```

```
2 hours and 15 minutes
```

Even or odd. An even number splits into 2 with nothing left over:

```
print(14 % 2)
print(7 % 2)
```

```
0
1
```

`% 2` gives `0` for even numbers and `1` for odd ones. Remember this; it's on the next page.

The last digit of a number. Splitting by 10, the leftover is the last digit. And `// 10` drops the last digit:

```
number = 1234
print(number % 10)
print(number // 10)
```

```
4
123
```

```
   1 2 3 | 4
   ──────   ─
     │      └── 1234 %  10  ->  4     (the last digit)
     └─────────  1234 // 10  ->  123   (everything except the last digit)
```

You'll use this pair to take numbers apart, digit by digit, later on.

Updating a box

You can put a new value in a box. Python works out the right side first, then puts the answer in the box:

```
score = 10
score = score + 1
print(score)
```

```
11
```

`score = score + 1` reads as "take what's in score, add 1, put it back in score". Because this is so common, there's a short way to write it:

```
score += 5     # same as  score = score + 5
print(score)
```

```
16
```

`-=`, `*=` and `//=` work the same way.

Rounding

`round(value, places)` rounds a float to that many places after the decimal point:

```
print(7 / 3)
print(round(7 / 3, 2))
```

```
2.3333333333333335
2.33
```

Without the second number, `round` gives a whole number: `round(7 / 3)` is `2`.

Try it

From the `learn` folder, run `examples/02-variables-and-numbers.py`:

```
python3 examples/02-variables-and-numbers.py
```

```
20 49.5 Tanvi False
<class 'int'>
<class 'float'>
<class 'str'>
<class 'bool'>
9
5
14
3.5
3
1
49
2 hours and 15 minutes
0
1
4
123
11
16
2.33
```

Then change things in it. Set `minutes` to `200` and `number` to `9876`. Guess the output before you run it.

Common mistakes

Adding text and a number.

```
print("5" + 3)
```

```
TypeError: can only concatenate str (not "int") to str
```

A **TypeError** means the kinds don't fit together. `"5"` is text (it has quotes), `3` is a number, and Python won't add text to a number. Remove the quotes: `5 + 3` is `8`.

Dividing by zero.

```
print(10 / 0)
```

```
ZeroDivisionError: division by zero
```

Same as in maths: you can't split something into zero groups. You get a `ZeroDivisionError` with `//` and `%` by zero too.

Using a box before making it. `print(total)` before any `total = ...` line gives `NameError: name 'total' is not defined`, the same error as a typo on page 1.

Check yourself

1. What do `17 // 5` and `17 % 5` give?
2. How can you tell if a number is even, using `%`?
3. What does `10 / 5` print, and why isn't it `2`?

Next

Next your programs will make choices: If and else.

Example script: `examples/02-variables-and-numbers.py`

Run it from the `learn` folder, then change things and run it again.

```python
# Page 2: Variables and numbers
# Run me with:  python3 02-variables-and-numbers.py

# A variable is a labelled box. The = puts a value in the box.
age = 20
price = 49.5
name = "Tanvi"
is_raining = False

print(age, price, name, is_raining)

# type() tells you what kind of value is in the box.
print(type(age))         # int: a whole number
print(type(price))       # float: a number with a decimal point
print(type(name))        # str: text
print(type(is_raining))  # bool: True or False

# The maths operators
print(7 + 2)    # 9
print(7 - 2)    # 5
print(7 * 2)    # 14
print(7 / 2)    # 3.5   ( / always gives a float)
print(7 // 2)   # 3     ( // whole times it fits, drops the rest)
print(7 % 2)    # 1     ( % what's left over)
print(7 ** 2)   # 49    ( ** to the power of)

# Minutes to hours and minutes
minutes = 135
print(minutes // 60, "hours and", minutes % 60, "minutes")

# Even or odd: an even number has nothing left over when split by 2
print(14 % 2)   # 0, so 14 is even
print(7 % 2)    # 1, so 7 is odd

# The last digit, and the number without its last digit
number = 1234
print(number % 10)    # 4
print(number // 10)   # 123

# Updating a box
score = 10
score = score + 1
print(score)    # 11
score += 5
print(score)    # 16

# Rounding
print(round(7 / 3, 2))   # 2.33

# Try changing things:
#   1. Set minutes to 200. What do you expect before you run it?
```

```
#   2. Set number to 9876. What are the last digit and the rest?
#   3. Is 1001 even or odd? Check with % 2.
```

LESSON 3

If and else

After this page you'll be able to make your program choose what to do, with `if`, `elif` and `else`.

Choices you already make

"If it's raining, take an umbrella. Otherwise, take sunglasses."

You ask a yes-or-no question (is it raining?), and the answer decides what you do next. Programs work the same way. They ask a question, get `True` or `False`, and pick a path.

Questions with True or False answers

On page 2 you met the **bool** type: `True` or `False`. A **comparison** is a question whose answer is a bool:

```
print(5 > 3)
print(5 < 3)
```

```
True
False
```

All the comparisons:

Write	Means	Example	Answer
`==`	is equal to	`4 == 4`	`True`
`!=`	is not equal to	`4 != 4`	`False`
`<`	is less than	`2 < 5`	`True`
`>`	is more than	`2 > 5`	`False`
`<=`	is less than or equal to	`3 <= 3`	`True`
`>=`	is more than or equal to	`2 >= 5`	`False`

They work on text too. Capitals count, so `"cat" == "Cat"` is `False`.

`=` and `==` are different

This is the one to be careful with.

- `=` puts a value in a box: `age = 20` ("age gets 20").
- `==` asks a question: `age == 20` ("is age equal to 20?"). The answer is `True` or `False`.

```
age = 20
print(age == 20)
print(age == 18)
```

```
True
False
```

Joining questions: `and`, `or`, `not`

- `and`: `True` only if **both** sides are true. "Over 18 and under 60."
- `or`: `True` if **at least one** side is true. "Weekend or holiday."
- `not`: flips the answer. `not True` is `False`.

```
age = 20
print(age >= 18 and age < 60)
print(age < 10 or age > 60)
print(not age > 18)
```

```
True
False
False
```

`if`: do this only when it's true

```
age = 20

if age >= 18:
    print("You can vote")

print("Done")
```

```
You can vote
Done
```

Look closely at the shape:

```
if age >= 18:               <- the question, then a colon
    print("You can vote")   <- 4 spaces in: runs only if the answer is True
print("Done")               <- back at the left edge: runs every time
```

- The line starts with `if`, then the question, then a **colon** `:`.
- The lines underneath are pushed in by 4 spaces. This is called **indentation**. Indented lines belong to the `if`, and run only when the answer is `True`.
- The first line back at the left edge is outside the `if`. It always runs.

Change `age` to `15` and run it. You'll only see `Done`.

`else`: otherwise, do this

```python
number = 7

if number % 2 == 0:
    print(number, "is even")
else:
    print(number, "is odd")
```

```
7 is odd
```

This is the even/odd check from page 2, now making a choice. `number % 2` is `1`, so `1 == 0` is `False`, so Python skips the `if` part and runs the `else` part. Exactly one of the two always runs.

`elif`: more than two paths

`elif` is short for "else if". Use it when there are more than two choices:

```python
marks = 72

if marks >= 90:
    print("Grade A")
elif marks >= 70:
    print("Grade B")
elif marks >= 50:
    print("Grade C")
else:
    print("Try again")
```

```
Grade B
```

Python checks from the top, one by one:

```
marks = 72

marks >= 90 ?   False  -> skip
marks >= 70 ?   True   -> print "Grade B", then skip everything below
marks >= 50 ?   (never checked)
else            (never reached)
```

Only the **first** true one runs. That's why the order matters: `72` is also `>= 50`, but Python stopped at `>= 70`.

Try it

From the `learn` folder, run `examples/03-if-else.py`:

```
python3 examples/03-if-else.py
```

```
True
False
True
True
False
False
Grade B
7 is odd
Done
```

Then set `marks` to `95`, then `50`, then `10`, and `number` to `0`. Predict each answer before you run it.

Common mistakes

Forgetting to indent.

```
age = 20
if age >= 18:
print("adult")
```

```
IndentationError: expected an indented block after 'if' statement on line 2
```

An **IndentationError** means the spaces at the start of a line are wrong. Here Python saw an `if` and expected the next line to be pushed in. Add 4 spaces before `print`.

Too many spaces is also an error. A line indented when nothing asked for it gives `IndentationError: unexpected indent`. In Python, spaces at the start of a line have meaning. Most editors add 4 spaces when you press Tab.

Using `=` instead of `==`.

```
if age = 18:
```

```
SyntaxError: invalid syntax. Maybe you meant '==' or ':=' instead of '='?
```

Python noticed you used "put in a box" where a question belongs. Use `==`. (Ignore the `:=` it mentions; you won't need it.)

Forgetting the colon. `if age >= 18` with no `:` at the end gives `SyntaxError: expected ':'`.

Check yourself

1. What's the difference between `x = 5` and `x == 5`?
2. With `marks = 95`, which grade prints? Why doesn't `Grade B` also print?
3. What does `10 > 3 and 2 > 5` give?

Next

Next you'll package steps into your own reusable recipes: Functions.

Example script: `examples/03-if-else.py`

Run it from the `learn` folder, then change things and run it again.

```python
# Page 3: If and else
# Run me with:  python3 03-if-else.py

# A comparison is a question. The answer is True or False.
print(5 > 3)      # True
print(5 == 3)     # False  ( == asks "are these equal?")
print(5 != 3)     # True   ( != asks "are these different?")

# and / or / not
age = 20
print(age >= 18 and age < 60)   # True: both are true
print(age < 10 or age > 60)     # False: neither is true
print(not age > 18)             # False: flips True to False

# if / elif / else: Python checks from the top and runs the first match.
marks = 72

if marks >= 90:
    print("Grade A")
elif marks >= 70:
    print("Grade B")
elif marks >= 50:
    print("Grade C")
else:
    print("Try again")

# Even or odd
number = 7

if number % 2 == 0:
    print(number, "is even")
else:
    print(number, "is odd")

# Indented lines belong to the if. The last line is not indented,
# so it runs every time, whatever the if decided.
print("Done")

# Try changing things:
#   1. Set marks to 95, then 50, then 10. Predict each grade first.
#   2. Set number to 0. Is 0 even?
#   3. Remove the 4 spaces before one print, run it, and read the error.
```

LESSON 4

Functions

After this page you'll be able to write a function that takes inputs and returns an answer, and you'll know why `return` is not the same as `print`.

A recipe card

Think of a recipe card for chai. It has a name ("Chai"), a list of what you need to bring (tea leaves, milk, sugar), and steps. Once the card is written, anyone can say "make chai" with whatever milk and sugar they have, and get a cup of chai back.

A **function** is a recipe card in Python. You write the steps once, give them a name, and then use them as often as you like with different inputs. Every LeetCode answer you write will be a function.

Writing one: `def`

```
def add(a, b):
    return a + b
```

Read it piece by piece:

```
def add(a, b):        <- def = "define a function". Its name is add.
    return a + b      <- the step: work out a + b and hand it back
```

- `def` means "I'm writing a new function".
- `add` is its name. Same naming rules as variables.
- `a` **and** `b` are its **parameters**: the empty boxes for the inputs. They get filled in each time the function is used.
- The **colon** and the **indented** lines work like `if` on page 3. Indented lines are the function's steps.
- `return` hands an answer back to whoever asked.

Running this file does nothing visible. Writing a recipe card doesn't make any chai. You have to use it.

Using one: calling

Using a function is called **calling** it. Write its name with the inputs in brackets:

```python
def add(a, b):
    return a + b

total = add(2, 3)
print(total)
print(add(10, 20))
```

```
5
30
```

When Python reaches `add(2, 3)`:

```
add(2, 3)
     |  |
     ▼  ▼
def add(a, b):        a = 2, b = 3
    return a + b      2 + 3 = 5, hand 5 back
     |
     ▼
total = 5             add(2, 3) is replaced by the 5 it returned
```

The values you pass in (`2` and `3`) are called **arguments**. They fill the parameters in order: first one into `a`, second into `b`.

You've been calling functions since page 1: `print(...)`, `type(...)` and `round(...)` are all functions that come with Python.

`print` and `return` are different

This is the most important part of the page. It trips up almost every beginner, and on LeetCode it's the difference between a right answer and a wrong one.

- `print` shows something on the screen, for a person to read. The program can't use it afterwards.
- `return` hands a value back to the code that called the function. The program can store it, do maths with it, or check it.

Think of asking a friend to add up a bill. If they **shout** the total across the room (print), you heard it, but you have nothing in your hand. If they **write it on a slip and give it to you** (return), you can keep it, add a tip to it, or pass it on.

Here are two functions that look almost the same:

```
def add_and_print(a, b):
    print(a + b)

def add_and_return(a, b):
    return a + b

x = add_and_print(2, 3)
y = add_and_return(2, 3)
print("x is", x)
print("y is", y)
```

```
5
x is None
y is 5
```

What happened, line by line:

1. `x = add_and_print(2, 3)` ran the function. It printed `5` on the screen. Then it ended without returning anything. So `x` got `None`.
2. `y = add_and_return(2, 3)` ran the function. It printed nothing. It returned `5`, so `y` got `5`.
3. The last two lines show what's in each box.

`None` is Python's word for "nothing". A function with no `return` line gives back `None`.

Now try to use the answers:

```
print(add_and_return(2, 3) * 10)
```

```
50
```

```
print(add_and_print(2, 3) * 10)
```

```
5
Traceback (most recent call last):
  ...
TypeError: unsupported operand type(s) for *: 'NoneType' and 'int'
```

The second one printed `5` (from inside the function), then crashed. It got `None` back, and `None * 10` makes no sense. The `5` on the screen looked like an answer, but the program never got it.

Why this matters for LeetCode. LeetCode doesn't read your screen. It calls your function and checks what it **returns**. If you `print` the right answer and don't `return` it, LeetCode gets `None` and marks you wrong. So:

- Use `return` for the answer.
- Use `print` only to look at things while you're working something out.

`return` ends the function

As soon as Python reaches a `return`, it hands the value back and leaves the function. Lines after it don't run:

```
def say_hi():
    return "hi"
    print("you'll never see this")

print(say_hi())
```

```
hi
```

This is useful with `if`.

Returning True or False

A function can return a bool. Here's the even/odd check from pages 2 and 3 as a function:

```
def is_even(n):
    if n % 2 == 0:
        return True
    return False

print(is_even(4))
print(is_even(7))
```

```
True
False
```

- For `4`: `4 % 2 == 0` is `True`, so it returns `True` and leaves. The last line never runs.
- For `7`: the `if` is `False`, so Python skips it and reaches `return False`.

You don't need an `else` here, because `return` already left the function. Many LeetCode problems ask "return true if ...", and they look exactly like this.

One function calling another

A function can use another function, the way a recipe for chai biscuits might say "first make chai (see the chai card)":

```
def is_even(n):
    if n % 2 == 0:
        return True
    return False

def is_odd(n):
    return not is_even(n)

print(is_odd(7))
```

```
True
```

`is_odd(7)` calls `is_even(7)`, which returns `False`, and `not False` is `True`.

Try it

From the `learn` folder, run `examples/04-functions.py`:

```
python3 examples/04-functions.py
```

```
5
30
50
5
x is None
y is 5
True
False
True
2 15
```

Then open it and:

1. Write a function `square(n)` that returns `n * n`. Print `square(9)`.

2. Change `add_and_return` to use `print` instead of `return`. What does `y is` show now?
3. Write `is_positive(n)` that returns `True` when `n` is more than `0`.

Common mistakes

Printing instead of returning. No error message, which makes it sneaky. The answer appears on screen, but the function gives back `None`. If you ever see `None` where you expected a value, look for a missing `return`.

Giving the wrong number of inputs.

```
def add(a, b):
    return a + b

print(add(2))
```

```
TypeError: add() missing 1 required positional argument: 'b'
```

`add` has two parameters, `a` and `b`, but got only one value. `2` went into `a`, and `b` had nothing. Pass one value for each parameter: `add(2, 3)`.

Calling a function before the `def` line. Python reads top to bottom, so the `def` must come first. Otherwise you'll get a `NameError`, like a typo.

Check yourself

1. A function has `print(a * b)` but no `return`. What does it give back?
2. Why does LeetCode need `return` and not `print`?
3. In `is_even`, why is there no `else` before `return False`?

Next

You now know enough Python for your first LeetCode problems. Next, see what a LeetCode problem looks like and solve one: How a LeetCode problem looks.

Example script: `examples/04-functions.py`

Run it from the `learn` folder, then change things and run it again.

```python
# Page 4: Functions
# Run me with:  python3 04-functions.py

# def makes a function. a and b are its parameters (the inputs).
# return hands the answer back to whoever called it.
def add(a, b):
    return a + b

# Calling the function: put real values in the brackets.
total = add(2, 3)
print(total)             # 5
print(add(10, 20))       # 30
print(add(2, 3) * 10)    # 50: the returned 5 can be used in more maths


# print versus return
def add_and_print(a, b):
    print(a + b)         # shows 5 on the screen, but gives nothing back

def add_and_return(a, b):
    return a + b         # gives 5 back, shows nothing

x = add_and_print(2, 3)  # this line prints 5
y = add_and_return(2, 3) # this line prints nothing
print("x is", x)         # x is None: nothing came back
print("y is", y)         # y is 5


# A function that returns True or False
def is_even(n):
    if n % 2 == 0:
        return True
    return False         # only reached when the if was not true

print(is_even(4))        # True
print(is_even(7))        # False


# One function can call another
def is_odd(n):
    return not is_even(n)

print(is_odd(7))         # True


# Minutes to "hours, minutes" using two small functions
def hours(minutes):
    return minutes // 60

def leftover_minutes(minutes):
    return minutes % 60

print(hours(135), leftover_minutes(135))   # 2 15
```

```
# Try changing things:
#   1. Write a function square(n) that returns n * n. Print square(9).
#   2. Change add_and_return to use print instead. What does "y is" show now?
#   3. Write is_positive(n) that returns True when n is bigger than 0.
```

LESSON 5

How a LeetCode problem looks

After this page you'll be able to read a LeetCode problem, understand the code it gives you, solve a practice file on your laptop, and submit it on LeetCode.

A workbook with an answer checker

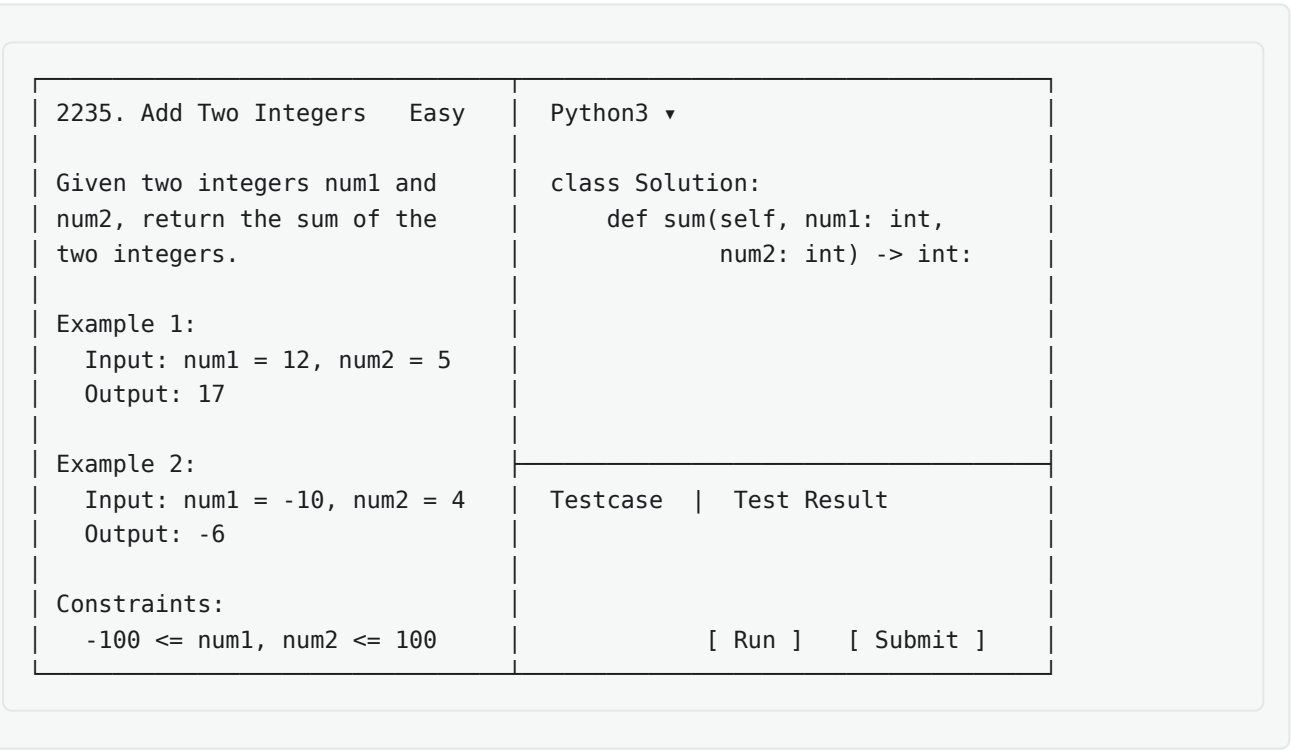
Remember maths workbooks with the answers at the back? You'd solve a question, then flip to the back to check.

LeetCode (leetcode.com) is a website full of programming questions, called **problems**, with a checker built in. You write a function, press a button, and LeetCode runs your function on lots of inputs and tells you if every answer was right. Companies use these problems in interviews, and they're a very good way to practise.

You know enough Python now to solve your first ones.

What a problem page looks like

Open 2235. Add Two Integers. The page has two halves:



On the left, the problem:

- **Description**: what your function gets and what it must return. Read it twice.
- **Examples**: some inputs and the right output for each. Work through one on paper before writing code.

- **Constraints**: the limits on the inputs. Here, both numbers are between -100 and 100. Later, constraints tell you things like "the list is never empty", which saves you worrying about it.

On the right, the code:

- A **language** menu. Pick **Python3** (not "Python", which is an old version).
- The **code editor**, with the start of a function already written for you. This is called the **template**. You fill in the inside.
- **Run** tries your code on the examples only. Use it as often as you like.
- **Submit** runs your code on all of LeetCode's hidden tests (often hundreds) and gives the final result.

Two words you'll see in problems: an **integer** is a whole number (an `int`), and an **array** is a list of values (you'll learn lists on page 7). Also, LeetCode writes `true` and `false` in small letters, because it shares the text with other languages. In Python they're always `True` and `False`.

`class Solution`: the box your function lives in

Here's the template again, the way it looks in a file:

```python
class Solution:
    def sum(self, num1: int, num2: int) -> int:
```

You know `def` from page 4. Two new things:

- `class Solution:` is a box that LeetCode puts your function in. Every problem has it. You don't need to understand classes to solve these problems. Leave that line exactly as it is, and write your function indented inside it.
- `self` is always the first parameter. LeetCode fills it in by itself. Ignore it for now. You never pass anything for it, and you won't use it in these problems.

So, with `self` set aside, this is a function named `sum` that takes two inputs, `num1` and `num2`. Exactly like `add(a, b)` on page 4.

Type hints: labels on the boxes

The `: int` and `-> int` parts are **type hints**. They're labels saying what kind of value goes in and what kind comes out:

```
def sum(self, num1: int, num2: int) -> int:
                    ────         ────     ────
                     │           │        └── it returns an int
                     │           └─────────── num2 is an int
                     └─────────────────────── num1 is an int
```

Python doesn't check them. They're there to tell **you** what to expect. Read them first, every time. `-> int` means your answer must be a whole number, `-> bool` means `True` or `False`, `-> str` means text.

Later problems use list types like this:

```
class Solution:
    def runningSum(self, nums: list[int]) -> list[int]:
```

- `list[int]` means "a list of whole numbers". So `nums` is a list of ints, and the function returns a list of ints.
- `list[str]` is a list of strings, and `list[list[int]]` is a list of lists (a grid; that's lesson 12).
- Older solutions you find online write `List[int]` with a capital L and a line `from typing import List` at the top. It means the same thing; that's the older way of writing it.

Solve your first problem on your laptop

Every problem in `../practice/` is a Python file with the template, some tests, and hints. Open `../practice/1-numbers-and-if/01-2235-add-two-integers.py`. It looks like this:

```
"""
2235. Add Two Integers  (Easy)
https://leetcode.com/problems/add-two-integers/

In plain words:
    You get two whole numbers, num1 and num2. Return their sum.
    The numbers can be negative.

Example:
    num1 = 12, num2 = 5   ->  17
    num1 = -10, num2 = 4  ->  -6

You'll need: + (page 2), return (page 4)

Run this file:  python3 01-2235-add-two-integers.py
"""


class Solution:
    def sum(self, num1: int, num2: int) -> int:
        # your code here
        pass


# ---------- tests: you don't need to change anything below ----------
...
```

- The top part, between the `"""` marks, explains the problem in plain words and says which pages you need.
- `pass` means "do nothing". It's a placeholder, because Python doesn't allow a function with no steps at all. You'll replace it.
- Below the tests line is code that checks your answer. You don't need to read or change it. (It uses things from later pages.)
- At the very bottom are **hints**. Read them only if you're stuck.

1. Run it before solving

In the terminal, from the `python-dsa` folder, go to the stage folder and run the file:

```
cd practice/1-numbers-and-if
python3 01-2235-add-two-integers.py
```

```
✗ (12, 5) -> got None but expected 17
✗ (-10, 4) -> got None but expected -6
✗ (0, 0) -> got None but expected 0
✗ (-3, -7) -> got None but expected -10
```

Each line is one test. Read the first one: with inputs `12` and `5`, your function returned `None` (nothing, because of `pass`), but the right answer is `17`. A ✗ means wrong. That's expected; you haven't written anything yet.

2. Write the answer

Replace the two lines `# your code here` and `pass` with:

```
class Solution:
    def sum(self, num1: int, num2: int) -> int:
        return num1 + num2
```

Keep the indentation: `return` sits 8 spaces in, inside `def`, which is inside `class`.

3. Run it again

```
python3 01-2235-add-two-integers.py
```

```
✓ (12, 5) -> 17
✓ (-10, 4) -> -6
✓ (0, 0) -> 0
✓ (-3, -7) -> -10
```

Every line is ✓. Solved.

What a wrong answer looks like

Try `print` instead of `return`, the trap from page 4:

```
        print(num1 + num2)
```

```
17
✗ (12, 5) -> got None but expected 17
-6
✗ (-10, 4) -> got None but expected -6
0
✗ (0, 0) -> got None but expected 0
-10
✗ (-3, -7) -> got None but expected -10
```

The right numbers appear on screen (`17`, `-6`, `0`, `-10`), but the tests still fail, because the function returned `None`. LeetCode would mark this wrong too. Put `return` back.

A wrong calculation looks like this, for example with `return num1 - num2`:

```
✗ (12, 5) -> got 7 but expected 17
✗ (-10, 4) -> got -14 but expected -6
✓ (0, 0) -> 0
✗ (-3, -7) -> got 4 but expected -10
```

"got 7 but expected 17" tells you exactly which input failed and what you returned. Work through that input by hand to find the mistake. Notice that `(0, 0)` passed by luck (`0 - 0` is also `0`). One ✓ doesn't mean the code is right; you need every line to be ✓.

Submit on LeetCode

Once every line shows ✓:

1. Open the problem on LeetCode (the link is at the top of the file) and sign in.
2. Check the language menu says **Python3**.

3. Copy only the `class Solution` part from your file: from `class Solution:` down to your `return` line. Not the tests, not the hints.
4. Paste it into LeetCode's editor, replacing what's there.
5. Press **Run**. It tries the examples. You should see your output match the expected output.
6. Press **Submit**.

You'll see one of these:

- **Accepted** (green): every hidden test passed. Done.
- **Wrong Answer**: one of the hidden tests failed. LeetCode shows that input, your output, and the expected output. Work that input through by hand, fix your code, and submit again.
- **Runtime Error**: your code crashed, with an error message like the ones you've seen. Read the last line, as on page 1.

Wrong answers are normal, and nobody sees them but you. Submitting several times is fine.

Try it

From the `learn` folder, run `examples/05-how-leetcode-looks.py`:

```
python3 examples/05-how-leetcode-looks.py
```

```
8
-6
✓ 4 -> 8
✓ -3 -> -6
✓ 0 -> 0
```

It's a made-up problem ("return `n` doubled") in the LeetCode shape, with a small checker built only from pages 1 to 4. Change `return n * 2` to `return n * 3` and see which lines turn into ✗. Then try `print(n * 2)` and look at what "got" says.

Common mistakes

Forgetting the indentation inside `class Solution`. If `def` isn't pushed in under `class Solution:`, you'll see:

```
IndentationError: expected an indented block after class definition on line 1
```

`def` goes 4 spaces in. The lines inside `def` go 8 spaces in.

Leaving `pass` and getting `None`. If every test says `got None`, your function isn't returning anything. Check that you replaced `pass` and used `return`, not `print`.

Renaming the function. LeetCode calls the function by its exact name (`sum` here). If you change it to `add`, LeetCode can't find it. Keep the name and the parameters exactly as given.

Check yourself

1. What's the difference between **Run** and **Submit** on LeetCode?
2. In `def isPowerOfTwo(self, n: int) -> bool:`, what goes in and what must come out?
3. Your file shows `got None but expected 5`. What's the most likely mistake?

Next

Time to practise. Solve the problems in **Stage 1** of the practice list (`leetcode-list.pdf`), in order. Add Two Integers is one of them, and you've already solved it above.

Example script: `examples/05-how-leetcode-looks.py`

Run it from the `learn` folder, then change things and run it again.

```python
# Page 5: How a LeetCode problem looks
# Run me with:  python3 05-how-leetcode-looks.py
#
# This is the shape every LeetCode answer has, with a small made-up
# problem: "You get a number n. Return n doubled."

class Solution:                              # the box LeetCode puts your function in
    def double(self, n: int) -> int:         # self: always first, ignore it for now
        return n * 2                         # n: int  means "n is a whole number"
                                             # -> int  means "it gives back a whole
number"


# How to call a function that lives inside the box:
# Solution() makes the box, .double(4) calls the function in it.
print(Solution().double(4))     # 8
print(Solution().double(-3))    # -6


# A small checker, built only from things you know (pages 1 to 4).
# The practice files do the same job with a few lines you'll learn later.
def check(n, expected):
    got = Solution().double(n)
    if got == expected:
        print("✓", n, "->", got)
    else:
        print("✗", n, "-> got", got, "but expected", expected)

check(4, 8)
check(-3, -6)
check(0, 0)

# Try changing things:
#   1. Change  return n * 2  to  return n * 3. Run it. Which lines show ✗?
#   2. Change it to  print(n * 2). What does "got" show now, and why?
#   3. Put  return n * 2  back so every line shows ✓.
```

LESSON 6

Strings

After this page you'll be able to pick letters out of text, cut pieces from it, and change it with Python's ready-made tools.

A necklace of letter beads

Think of a word as a necklace of beads, one letter on each bead. The beads are in a fixed order, and you can count them: first bead, second bead, and so on.

In Python, text is called a **string** (a string of letters, like a string of beads). You've met it already: it's the `str` type, text inside quotes.

```
word = "python"
greeting = 'hello'     # single quotes work too
```

Index: one letter at a time

Every letter has a position number, called its **index**. Python starts counting at **0**, not 1.

```
word = "python"

  p    y    t    h    o    n
  0    1    2    3    4    5      <- index from the front
 -6   -5   -4   -3   -2   -1      <- index from the back
```

Put the index in square brackets to get that letter:

```
word = "python"
print(word[0])
print(word[5])
print(word[-1])
print(word[-2])
print(len(word))
```

You'll see:

```
p
n
n
o
6
```

- `word[0]` is the first letter.
- A **negative index** counts from the back: `word[-1]` is always the last letter, `word[-2]` the one before it.
- `len()` (short for length) tells you how many letters there are. Spaces count as letters too.

Notice: the last index is `5`, but the length is `6`. The last index is always `len(word) - 1`, because counting starts at 0.

Slicing: cut out a piece

A **slice** is a piece of the string. You write `[start:stop]`. You get the letters from `start` up to, **but not including**, `stop`.

```
word = "python"
print(word[0:3])
print(word[2:5])
print(word[2:])
print(word[:4])
print(word[::-1])
```

You'll see:

```
pyt
tho
thon
pyth
nohtyp
```

- `word[0:3]` gives index 0, 1, 2. Not 3.
- Leave out the start and Python starts at the beginning. Leave out the stop and it goes to the end.
- `word[::-1]` is the string **backwards**. The third number is the **step**: how far to move each time. `-1` means "walk backwards, one letter at a time". You'll use this one a lot.

Is this piece inside? `in`

`in` asks "is this text somewhere in that text?" The answer is `True` or `False`.

```
word = "python"
print("th" in word)
print("z" in word)
```

```
True
False
```

Methods: tools that come with every string

A **method** is a tool that belongs to a value. You use it with a dot: `value.method()`. Every string comes with these:

```
name = "  Tanvi Kumar  "
print(name.lower())
print(name.upper())
print(name.strip())
```

```
  tanvi kumar
  TANVI KUMAR
Tanvi Kumar
```

- `lower()` makes every letter small. `upper()` makes every letter capital.
- `strip()` removes spaces from both ends (not from the middle).

```
ip = "1.1.1.1"
print(ip.replace(".", "[.]"))
print("hello".find("l"))
print("hello".find("z"))
```

```
1[.]1[.]1[.]1
2
-1
```

- `replace(old, new)` swaps every `old` piece for `new`.
- `find(piece)` gives the index where `piece` first appears. If it isn't there, you get `-1`.

Two methods answer yes/no questions about the letters:

```
print("2024".isdigit())
print("20a4".isdigit())
print("abc123".isalnum())
print("a,b".isalnum())
```

```
True
False
True
False
```

- `isdigit()`: is every character a digit (0 to 9)?
- `isalnum()`: is every character a letter or a digit? ("alnum" is short for alphabet + number.) A comma or a space makes it `False`.

Split a sentence into words, and join them back

`split()` cuts a string into words wherever there are spaces.

```
sentence = "fly me   to the moon"
words = sentence.split()
print(words)
print(words[-1])
print(len(words))
```

```
['fly', 'me', 'to', 'the', 'moon']
moon
5
```

The thing in square brackets is a **list**: many values kept in order. Lists are the next page. For now, notice that the same index tricks work on it: `words[-1]` is the last word, and `len(words)` counts the words.

Also notice: the extra spaces between "me" and "to" disappeared. `split()` with nothing in the brackets handles any number of spaces.

You can split on something else too, like a comma: `"a,b,c".split(",")` gives `['a', 'b', 'c']`.

`join()` does the opposite. It glues a list of strings together, with the string before the dot in between:

```
print(" ".join(words))
print("-".join(words))
```

```
fly me to the moon
fly-me-to-the-moon
```

You can't change a string, only make a new one

You can't swap one bead on a string. Try:

```
word = "python"
word[0] = "j"
```

```
TypeError: 'str' object does not support item assignment
```

Strings can't be changed in place. Instead, build a **new** string from pieces:

```
word = "python"
new_word = "j" + word[1:]
print(new_word)
print(word)
```

```
jython
python
```

The same is true for methods. `lower()`, `replace()` and the others give you a new string. The old one stays as it was:

```
s = "hello"
s.upper()
print(s)
s = s.upper()
print(s)
```

```
hello
HELLO
```

The first `s.upper()` made `"HELLO"` and then threw it away, because nothing kept it. To keep it, put it back in the box: `s = s.upper()`.

Numbers to text and back: `str()` and `int()`

- `str(x)` turns a number into text.
- `int(x)` turns text made of digits into a number.

Why would you want this? Text has tools that numbers don't. You can't reverse the number `123`, but you can reverse the text `"123"`:

```
n = 123
print(str(n))
print(str(n)[::-1])
print(int("42") + 1)
print(len(str(98765)))
```

```
123
321
43
5
```

`len(str(98765))` is a quick way to count the digits in a number.

f-strings: put values inside text

An **f-string** is a string with an `f` before the quotes. Anything inside `{ }` is worked out and dropped into the text.

```
name = "Tanvi"
age = 20
print(f"{name} is {age} years old")
print(f"{name} has {len(name)} letters")
print(f"Next year: {age + 1}")
```

```
Tanvi is 20 years old
Tanvi has 5 letters
Next year: 21
```

This is the easiest way to mix text and numbers in one line.

Try it

Run `examples/06-strings.py`:

```
python3 examples/06-strings.py
```

Then change the word at the top to your own name and run it again. Predict each line before you look.

Common mistakes

Asking for an index that doesn't exist. `"python"` has indexes 0 to 5, so:

```
word = "python"
print(word[6])
```

```
IndexError: string index out of range
```

It means "there's no letter at that position". The last index is `len(word) - 1`.

Adding text and a number.

```
print("Age: " + 20)
```

```
TypeError: can only concatenate str (not "int") to str
```

Concatenate means "join together". Python won't join text and a number with `+`. Use `"Age: " + str(20)`, or an f-string: `f"Age: {20}"`.

Forgetting to keep the new string. `s.replace("a", "b")` on its own line changes nothing you can see, and there's no error. Write `s = s.replace("a", "b")`.

Check yourself

1. `fruit = "banana"`. What are `fruit[1]`, `fruit[-2]` and `fruit[1:4]`?
2. What does `str(-121)[::-1]` give? Is it the same as `str(-121)`?
3. What does `"a b c".split()` give, and how would you join it back with no spaces at all?

Next

Time to use this on real problems: **Stage 2** in the LeetCode list (`leetcode-list.pdf`).

Example script: `examples/06-strings.py`

Run it from the `learn` folder, then change things and run it again.

```python
# Page 6: Strings
# Run me with:  python3 06-strings.py

word = "python"

# Index: position numbers start at 0. Negative ones count from the back.
print(word[0])        # p   (first letter)
print(word[-1])       # n   (last letter)
print(len(word))      # 6   (how many letters)

# Slicing: [start:stop] stops right BEFORE stop
print(word[0:3])      # pyt
print(word[2:])       # thon
print(word[::-1])     # nohtyp  (backwards)

# in: is this piece inside?
print("th" in word)   # True

# Methods give you a NEW string. The old one doesn't change.
name = "  Tanvi Kumar  "
print(name.strip())             # Tanvi Kumar
print(name.strip().lower())     # tanvi kumar  (one tool after another)
print("1.1.1.1".replace(".", "[.]"))   # 1[.]1[.]1[.]1
print("hello".find("l"))        # 2   (first place it appears)
print("hello".find("z"))        # -1  (not there)
print("2024".isdigit())         # True
print("hi!".isalnum())          # False (! is not a letter or digit)

# split makes a list of words. join glues them back.
words = "fly me   to the moon".split()
print(words)              # ['fly', 'me', 'to', 'the', 'moon']
print(words[-1])          # moon (the last word)
print(len(words[-1]))     # 4    (letters in the last word)
print("-".join(words))    # fly-me-to-the-moon

# Strings can't be changed in place. Build a new one instead.
new_word = "j" + word[1:]
print(new_word)           # jython

# Numbers to text and back
n = 1234
print(str(n)[::-1])       # 4321
print(len(str(n)))        # 4   (number of digits)
print(int("42") + 1)      # 43

# f-strings: values inside text
print(f"{word} has {len(word)} letters, backwards it's {word[::-1]}")

# Try changing things:
#   1. Set word to your own name. Predict every line before you run it.
#   2. Is "level" the same backwards? Print  "level" == "level"[::-1]
#   3. Print the first word of "fly me to the moon" in capital letters.
```

LESSON 7

Lists

After this page you'll be able to keep many values in one place, in order, and change them.

A shelf of numbered boxes

Picture a shelf with a row of boxes. Each box holds one thing, and each box has a number painted under it, starting at 0. You can look in any box, swap what's inside, add a box at the end, or take one away.

That shelf is a **list**. Most LeetCode problems hand you a list, usually called `nums`. When the problem says `nums: list[int]`, it means "a list of whole numbers".

Making a list, and reading from it

Square brackets, with commas between the items:

```
nums = [4, 8, 15, 16, 23]
```

```
nums = [4, 8, 15, 16, 23]

 +----+----+----+----+----+
 |  4 |  8 | 15 | 16 | 23 |
 +----+----+----+----+----+
    0    1    2    3    4      <- index
   -5   -4   -3   -2   -1      <- negative index
```

Everything you learned about strings works here: index, negative index, `len`, slicing.

```
nums = [4, 8, 15, 16, 23]
print(nums[0])
print(nums[-1])
print(len(nums))
print(nums[1:3])
print(nums[::-1])
```

You'll see:

```
4
23
5
[8, 15]
[23, 16, 15, 8, 4]
```

A list can hold any kind of value: numbers, strings, `True` / `False`, even other lists. The `split()` from the last page gave you a list of strings.

Changing an item

Here's the big difference from strings. A list **can** be changed in place. Put a new value in a box:

```
nums = [4, 8, 15, 16, 23]
nums[0] = 99
print(nums)
```

```
[99, 8, 15, 16, 23]
```

Adding and taking away: `append`, `insert`, `pop`

```
fruits = ["apple", "mango"]
fruits.append("kiwi")
print(fruits)
fruits.insert(0, "banana")
print(fruits)
```

```
['apple', 'mango', 'kiwi']
['banana', 'apple', 'mango', 'kiwi']
```

- `append(x)` adds `x` at the end. This is the one you'll use most.
- `insert(i, x)` puts `x` at index `i` and pushes everything after it one box along.

```
last = fruits.pop()
print(last)
print(fruits)
first = fruits.pop(0)
print(first)
print(fruits)
```

```
kiwi
['banana', 'apple', 'mango']
banana
['apple', 'mango']
```

- `pop()` takes the last item off the list and gives it to you.
- `pop(i)` takes the item at index `i`.

Is it on the shelf? `in`

```
print("mango" in fruits)
print("grape" in fruits)
```

```
True
False
```

Joining and repeating lists: `+` and `*`

```
print([1, 2] + [3, 4])
print([0] * 5)
print([1, 2] * 3)
```

```
[1, 2, 3, 4]
[0, 0, 0, 0, 0]
[1, 2, 1, 2, 1, 2]
```

`+` puts two lists end to end. `*` repeats a list. `[0] * 5` is a handy way to make a list of five zeros.

Sorting: `sort()` and `sorted()`

There are two ways to sort, and the difference matters.

`sort()` sorts the list itself. The old order is gone.

```
marks = [70, 45, 90, 60]
marks.sort()
print(marks)
```

```
[45, 60, 70, 90]
```

`sorted()` leaves the list alone and gives you a **new**, sorted list.

```
marks = [70, 45, 90, 60]
ordered = sorted(marks)
print(ordered)
print(marks)
```

```
[45, 60, 70, 90]
[70, 45, 90, 60]
```

For biggest first, add `reverse=True` in the brackets: `sorted(marks, reverse=True)` gives `[90, 70, 60, 45]`. The same works with `marks.sort(reverse=True)`.

The trap: `sort()` changes the list but gives back nothing. Remember `None` from page 4, the value of "nothing came back"?

```
marks = [70, 45, 90, 60]
result = marks.sort()
print(result)
```

```
None
```

So never write `marks = marks.sort()`. That sorts the list, then throws it away and puts `None` in the box. Either write `marks.sort()` on its own line, or `marks = sorted(marks)`.

Quick answers: `sum`, `max`, `min`, `reverse`

```
marks = [70, 45, 90, 60]
print(sum(marks))
print(max(marks))
print(min(marks))
marks.reverse()
print(marks)
```

```
265
90
45
[60, 90, 45, 70]
```

- `sum` adds everything up. `max` gives the biggest, `min` the smallest.
- `reverse()` flips the list itself, back to front. Like `sort()`, it changes the list and returns `None`.

Lists inside lists

A list can hold other lists. Think of a shelf where each box holds a smaller shelf.

```
accounts = [[1, 2, 3], [3, 2, 1], [5, 5, 5]]
print(accounts[1])
print(accounts[1][2])
print(len(accounts))
print(sum(accounts[2]))
```

```
[3, 2, 1]
1
3
15
```

```
accounts =  [ [1, 2, 3],  [3, 2, 1],  [5, 5, 5] ]
                0            1            2

accounts[1]     ->  [3, 2, 1]
accounts[1][2]  ->  [3, 2, 1][2]  ->  1
```

Read `accounts[1][2]` left to right: first take box 1 (a whole list), then box 2 inside that.

A function that changes a list changes it for everyone

When you pass a list to a function, the function gets the **same** shelf, not a copy. If it changes the shelf, the caller sees the change.

```
def double_first(nums):
    nums[0] = nums[0] * 2

my_list = [5, 6, 7]
double_first(my_list)
print(my_list)
```

```
[10, 6, 7]
```

The function returned nothing, but `my_list` changed.

This matters on LeetCode. Some problems say **"Do not return anything, modify nums in-place"**. **In place** means "change the list you were given, don't make a new one". LeetCode then looks at your list, not at what you returned.

One catch. Changing the boxes changes the caller's list. Putting a whole new list in the name does **not**:

```
def reverse_wrong(nums):
    nums = nums[::-1]     # a NEW list, only inside this function

def reverse_right(nums):
    nums.reverse()        # changes the list itself

a = [1, 2, 3]
reverse_wrong(a)
print(a)
reverse_right(a)
print(a)
```

```
[1, 2, 3]
[3, 2, 1]
```

`nums = ...` inside a function makes the name `nums` point at a new shelf. The caller's shelf is untouched. For in-place problems, use things that change the list itself: `nums[i] = ...`, `append`, `pop`, `sort()`, `reverse()`.

Try it

Run `examples/07-lists.py`:

```
python3 examples/07-lists.py
```

Then add your own list of five numbers and print its biggest, smallest and total.

Common mistakes

An index past the end. `[4, 8, 15]` has indexes 0, 1, 2:

```
nums = [4, 8, 15]
print(nums[3])
```

```
IndexError: list index out of range
```

It means "there's no box with that number". The last index is `len(nums) - 1`. This is the error you'll see most often with lists.

Keeping the result of `sort()`.

```
marks = [3, 1]
marks = marks.sort()
print(marks[0])
```

```
TypeError: 'NoneType' object is not subscriptable
```

`marks` is now `None`, and you can't use `[0]` on `None`. ("Subscriptable" means "you can use square brackets on it".)

Check yourself

1. `nums = [2, 7, 11, 15]`. What are `nums[-1]` and `nums[1:3]`?
2. After `x = [3, 1, 2]` and `y = sorted(x)`, what is in `x`, and what is in `y`?
3. `grid = [[1, 2], [3, 4]]`. How do you get the `3`?

Next

Practise with lists: **Stage 3** in the LeetCode list (`leetcode-list.pdf`).

Example script: `examples/07-lists.py`

Run it from the `learn` folder, then change things and run it again.

```python
# Page 7: Lists
# Run me with:  python3 07-lists.py

nums = [4, 8, 15, 16, 23]

# Reading: same as strings
print(nums[0])        # 4
print(nums[-1])       # 23
print(len(nums))      # 5
print(nums[1:3])      # [8, 15]

# Changing a box (strings can't do this, lists can)
nums[0] = 99
print(nums)           # [99, 8, 15, 16, 23]

# Adding and taking away
nums.append(42)       # add at the end
print(nums)           # [99, 8, 15, 16, 23, 42]
top = nums.pop()      # take the last one off
print(top, nums)      # 42 [99, 8, 15, 16, 23]
nums.insert(1, 7)     # put 7 at index 1
print(nums)           # [99, 7, 8, 15, 16, 23]
print(15 in nums)     # True

# + joins lists, * repeats them
print([1, 2] + [3])   # [1, 2, 3]
print([0] * 4)        # [0, 0, 0, 0]

# sort() changes the list. sorted() gives a new one.
marks = [70, 45, 90, 60]
print(sorted(marks))  # [45, 60, 70, 90]
print(marks)          # [70, 45, 90, 60]  (unchanged)
marks.sort()
print(marks)          # [45, 60, 70, 90]  (changed)
print(marks.sort())   # None  (sort gives nothing back)

# Quick answers
print(sum(marks), max(marks), min(marks))   # 265 90 45

# The two biggest are at the end after sorting
print(marks[-1], marks[-2])                 # 90 70

# Lists inside lists
accounts = [[1, 2, 3], [3, 2, 1], [5, 5, 5]]
print(accounts[1][2])     # 1   (box 1, then box 2 inside it)
print(sum(accounts[2]))   # 15


# A function that changes a list changes it for the caller too
def add_zero(nums):
    nums.append(0)

my_list = [1, 2, 3]
add_zero(my_list)
```

```
print(my_list)            # [1, 2, 3, 0]

# Try changing things:
#   1. Make a list of your three favourite foods. Add a fourth with append.
#   2. Make the list [1, 2, 3] twice in a row: [1, 2, 3, 1, 2, 3]. Two ways!
#   3. Write  marks = marks.sort()  then  print(marks). What do you get, and why?
```

LESSON 8

Loops

After this page you'll be able to repeat work with `for` and `while`, and you'll know the four loop patterns that solve most easy LeetCode problems.

This is the most important page so far. Take it slowly, one section at a time, and run every example.

A playlist, one song at a time

A music app plays a playlist like this: take the first song, play it. Take the next song, play it. Keep going until there are no songs left.

A **loop** is the same idea in code: "for each thing in this group, do this". You write the work once, and Python repeats it for every item.

`for`: do something for each item

```
songs = ["Kesariya", "Tum Hi Ho", "Ilahi"]
for song in songs:
    print("Now playing:", song)
print("Playlist over")
```

You'll see:

```
Now playing: Kesariya
Now playing: Tum Hi Ho
Now playing: Ilahi
Playlist over
```

How to read it:

1. `for song in songs:` means "take each item of `songs`, one at a time, and call it `song`".
2. The indented lines under it are the **body** of the loop. They run once per item, like the body of an `if`.
3. The first time round, `song` is `"Kesariya"`. The second time, `"Tum Hi Ho"`. Then `"Ilahi"`.
4. When there are no items left, the loop ends and Python carries on with the first line that isn't indented.

Each time round the loop is called an **iteration**. You can pick any name instead of `song`. Pick one that says what the item is.

Looping over a string

A string is a row of letters, so a `for` loop walks through it letter by letter:

```
for letter in "cat":
    print(letter)
```

```
c
a
t
```

`range`: counting numbers

Sometimes you don't have a list; you want the numbers 0, 1, 2, … `range` makes them.

```
print(list(range(5)))
print(list(range(2, 6)))
print(list(range(0, 10, 2)))
print(list(range(4, -1, -1)))
```

```
[0, 1, 2, 3, 4]
[2, 3, 4, 5]
[0, 2, 4, 6, 8]
[4, 3, 2, 1, 0]
```

(`list(...)` turns the range into a list, only so you can see the numbers. In a loop you don't need it.)

- `range(n)`: 0 up to n, but **not** n. So `range(5)` is five numbers, 0 to 4.
- `range(a, b)`: from a up to, but not including, b. Same rule as slicing.
- `range(a, b, step)`: jump by `step` each time. A negative step counts down. `range(4, -1, -1)` counts 4, 3, 2, 1, 0 (it stops before -1).

In a loop:

```
for i in range(3):
    print("Hello", i)
```

```
Hello 0
Hello 1
Hello 2
```

`i` is the usual name for a counting number like this (short for "index").

Index and item: `range(len(nums))` and `enumerate`

Looping with `for n in nums` gives you each **item**, but not its position. Sometimes you need the position too: to change `nums[i]`, to look at the next item `nums[i + 1]`, or to return where something was.

`range(len(nums))` gives you every index of the list:

```python
nums = [10, 20, 30]
for i in range(len(nums)):
    print(i, nums[i])
```

```
0 10
1 20
2 30
```

`len(nums)` is 3, so `range(3)` gives 0, 1, 2: exactly the indexes.

`enumerate` does the same job more neatly. It hands you two things each time round, the index and the item, and you give each one a name:

```python
nums = [10, 20, 30]
for i, n in enumerate(nums):
    print(i, n)
```

```
0 10
1 20
2 30
```

Use whichever reads more clearly to you.

The four loop patterns

Almost every easy LeetCode problem is one of these four shapes, or two of them together. Learn the shapes and you'll recognise them everywhere.

Each one has the same three parts: **set up** a variable before the loop, **update** it inside the loop, **use** it after the loop.

1. Build a result list

Start with an empty list, `append` to it each time round.

```
nums = [1, 2, 3, 4]
doubled = []
for n in nums:
    doubled.append(n * 2)
print(doubled)
```

```
[2, 4, 6, 8]
```

The same works for building a string: start with `""` and add to it.

```
word = "hello"
shout = ""
for ch in word:
    shout += ch.upper()
print(shout)
```

```
HELLO
```

2. Running total

Start at 0, add each item.

```
nums = [3, 1, 4, 1, 5]
total = 0
for n in nums:
    total += n
    print("added", n, "-> total is", total)
print(total)
```

```
added 3 -> total is 3
added 1 -> total is 4
added 4 -> total is 8
added 1 -> total is 9
added 5 -> total is 14
14
```

(`total += n` is short for `total = total + n`, from page 2.) `sum(nums)` gives the same answer, but this pattern lets you see the total *at each step*, and that's what some problems want.

3. Counter

Start at 0, add 1 each time something is true.

```python
nums = [3, 8, 5, 6, 2]
count = 0
for n in nums:
    if n % 2 == 0:
        count += 1
print(count)
```

```
3
```

Three even numbers: 8, 6 and 2.

4. Best so far

Start with the first item as the "best", and replace it whenever you meet something better.

```python
nums = [3, 8, 5, 9, 2]
best = nums[0]
for n in nums:
    if n > best:
        best = n
print(best)
```

```
9
```

```
n:      3    8    5    9    2
best:   3    8    8    9    9     <- best after looking at n
```

`max(nums)` does this for you. You still need the pattern, because problems often want the best of something you work out as you go, not the best item in the list. "Best" can also mean smallest: flip `>` to `<`.

`while`: repeat as long as something is true

A `for` loop goes through a group of things. A `while` loop keeps going **as long as** a condition is `True`. Like stirring tea *while* the sugar hasn't dissolved: you don't know in advance how many stirs.

Halve a number until it reaches zero, and count the steps:

```python
n = 20
steps = 0
while n > 0:
    n = n // 2
    steps += 1
    print("n is now", n)
print("steps:", steps)
```

```
n is now 10
n is now 5
n is now 2
n is now 1
n is now 0
steps: 5
```

Python checks `n > 0` before each iteration. When `n` reaches 0, the check is `False` and the loop stops.

Taking a number apart, digit by digit

From page 2: `% 10` gives the last digit, and `// 10` removes it. Put those in a `while` loop and you get every digit:

```python
n = 472
while n > 0:
    digit = n % 10
    print("digit:", digit)
    n = n // 10
```

```
digit: 2
digit: 7
digit: 4
```

```
 n      n % 10   n // 10
 472    2        47
 47     7        4
 4      4        0      <- n is 0, loop stops
```

The digits come out from the back. You'll use this whenever a problem talks about "the digits of a number".

Careful: if the condition never becomes `False`, the loop runs forever. If your program seems stuck, press **Ctrl+C** to stop it, then check that something inside the loop moves you towards the end (here, `n = n // 2`).

`break` and `continue`

- `break` stops the loop straight away.
- `continue` skips the rest of this iteration and moves on to the next item.

```python
nums = [4, 7, 9, -2, 5]
for n in nums:
    if n < 0:
        print("Found a negative:", n)
        break
    print("checked", n)
```

```
checked 4
checked 7
checked 9
Found a negative: -2
```

The `5` is never checked. Once you've found what you were looking for, there's no need to keep going.

```python
for n in [1, 2, 3, 4, 5]:
    if n % 2 == 0:
        continue
    print(n)
```

```
1
3
5
```

For even numbers, `continue` jumps back to the top before the `print`.

A loop inside a loop: every pair

Some problems need every **pair** of items: item 0 with item 1, item 0 with item 2, and so on. Put one loop inside another. Start the inner loop one step after the outer one, so `j` is always bigger than `i`:

```python
nums = [5, 6, 7]
for i in range(len(nums)):
    for j in range(i + 1, len(nums)):
        print(i, j, "->", nums[i], nums[j])
```

```
0 1 -> 5 6
0 2 -> 5 7
1 2 -> 6 7
```

Starting `j` at `i + 1` means you never pair an item with itself, and never count the same pair twice (0 with 1, and then 1 with 0).

A shorter way: list comprehensions

Pattern 1 (build a result list) is so common that Python has a one-line way to write it, called a **list comprehension**:

```python
nums = [1, 2, 3, 4]
print([n * 2 for n in nums])
print([n for n in nums if n % 2 == 0])
```

```
[2, 4, 6, 8]
[2, 4]
```

Read the first one as "`n * 2`, for each `n` in `nums`". The second keeps only the items where the `if` is true.

You'll see these in other people's solutions. Write the long way until it feels easy; then try the short way.

Try it

Run `examples/08-loops.py`:

```
python3 examples/08-loops.py
```

It has all four patterns, `while`, `break` and pairs. Change the lists and predict the output before you run it.

Common mistakes

Forgetting to indent the body.

```
for n in [1, 2]:
print(n)
```

```
IndentationError: expected an indented block after 'for' statement on line 1
```

The lines that belong to the loop must be indented (4 spaces), like under an `if`.

Using the item as if it were an index.

```
nums = [10, 20, 30]
for n in nums:
    print(nums[n])
```

```
IndexError: list index out of range
```

`n` is the item (10, 20, 30), not the position. `nums[10]` doesn't exist. Either print `n`, or loop with `range(len(nums))` if you need positions.

Looking one step too far. With `for i in range(len(nums))`, the last `i` is the last index. So `nums[i + 1]` goes past the end on the last iteration, and you get the same `IndexError`. If you need `nums[i + 1]`, loop with `range(len(nums) - 1)`.

Check yourself

1. Which numbers does `range(1, 10, 3)` give?
2. Write the four lines that count how many letters in `"banana"` are `"a"`. Which pattern is it?
3. What does this print, and why does it stop?

```
python n = 1 while n < 20: n = n * 3 print(n)
```

Next

This is the biggest practice stage, and where most of your learning happens: **Stage 4** in the LeetCode list (`leetcode-list.pdf`).

Example script: `examples/08-loops.py`

Run it from the `learn` folder, then change things and run it again.

```python
# Page 8: Loops
# Run me with:  python3 08-loops.py

nums = [3, 8, 5, 9, 2]

# for: once per item
for n in nums:
    print("item:", n)

# range(len(nums)) gives the indexes 0, 1, 2, ...
for i in range(len(nums)):
    print("index", i, "holds", nums[i])

# enumerate gives index and item together
for i, n in enumerate(nums):
    print(i, n)

# Counting down with a negative step: 4, 3, 2, 1, 0
for i in range(len(nums) - 1, -1, -1):
    print("from the back:", nums[i])

# --- The four patterns ---

# 1. Build a result list
doubled = []
for n in nums:
    doubled.append(n * 2)
print("doubled:", doubled)            # [6, 16, 10, 18, 4]

# 2. Running total
total = 0
for n in nums:
    total += n
print("total:", total)                # 27

# 3. Counter
evens = 0
for n in nums:
    if n % 2 == 0:
        evens += 1
print("even numbers:", evens)         # 2

# 4. Best so far
best = nums[0]
for n in nums:
    if n > best:
        best = n
print("biggest:", best)               # 9

# --- while ---

# Take a number apart, digit by digit
number = 472
digit_sum = 0
```

```
while number > 0:
    digit = number % 10      # last digit
    digit_sum += digit
    number = number // 10    # drop the last digit
print("sum of digits of 472:", digit_sum)   # 13

# break: stop as soon as you find it
for n in nums:
    if n > 6:
        print("first number above 6:", n)   # 8
        break

# continue: skip this one, go to the next
for n in nums:
    if n % 2 == 0:
        continue
    print("odd:", n)                        # 3, 5, 9

# Every pair (i before j)
pairs = 0
for i in range(len(nums)):
    for j in range(i + 1, len(nums)):
        pairs += 1
print("number of pairs:", pairs)            # 10

# The short way to build a list
print([n * 10 for n in nums])               # [30, 80, 50, 90, 20]
print([n for n in nums if n > 4])           # [8, 5, 9]

# Try changing things:
#   1. Count how many letters in "banana" are "a" (pattern 3, loop over a string).
#   2. Change pattern 4 to find the smallest number instead.
#   3. Count the steps to halve 100 down to 0 with a while loop.
```

LESSON 9

Dictionaries

After this page you'll be able to store "this goes with that" pairs, count things, and remember where you saw something.

The contacts app on your phone

Your phone's contacts app doesn't store numbers in a numbered row. You don't think "Mum is contact number 37". You type **Mum**, and the phone finds her number straight away.

Each entry is a pair: a **name** and the **number** that goes with it. The name is how you look things up.

Python has the same thing. It's called a **dictionary** (`dict` for short). You'll also hear it called a **map** or a **hash map**, because it maps each name to its value. All three words mean the same thing here.

- The thing you look up by (the name) is the **key**.
- The thing you get back (the number) is the **value**.
- A key and its value together are a **key-value pair**.

Making one and reading from it

Curly brackets. Each pair is written `key: value`, with commas between pairs.

```
contacts = {"Mum": "98100 11111", "Vikash": "98200 22222"}
print(contacts["Mum"])
print(len(contacts))
```

You'll see:

```
98100 11111
2
```

- `contacts["Mum"]` looks up the key `"Mum"` and gives you its value. Same square brackets as a list, but you put a **key** inside, not a position.
- `len` counts the pairs.

The lookup is fast, however big the dictionary gets. Your phone doesn't read every contact to find Mum, and neither does Python. That speed is why dictionaries show up in so many LeetCode answers.

Keys are usually strings or numbers. Values can be anything: numbers, strings, lists.

A key that isn't there: `get`

If you ask for a key that doesn't exist, Python stops with an error:

```
print(contacts["Riya"])
```

```
KeyError: 'Riya'
```

`get` is the safe way to ask. If the key isn't there, it gives you `None`, or a default value that you choose:

```
print(contacts.get("Mum"))
print(contacts.get("Riya"))
print(contacts.get("Riya", "not saved"))
```

```
98100 11111
None
not saved
```

Remember `.get(key, 0)`. You'll use it for counting in a moment.

Adding and changing

Put a value in with `=`. If the key is new, the pair is added. If the key is already there, its value is replaced.

```
contacts["Riya"] = "98300 33333"     # new key: added
contacts["Mum"] = "98100 99999"      # old key: updated
print(contacts)
```

```
{'Mum': '98100 99999', 'Vikash': '98200 22222', 'Riya': '98300 33333'}
```

A dictionary can't hold the same key twice. Like your phone, there's one entry per name.

Is this key in it? `in`

```
print("Riya" in contacts)
print("98300 33333" in contacts)
```

```
True
False
```

`in` checks the **keys** only. `"98300 33333"` is a value, not a key, so the answer is `False`.

Looping over a dictionary

A plain `for` loop gives you each key:

```
for name in contacts:
    print(name)
```

```
Mum
Vikash
Riya
```

`.items()` gives you each key and its value together. Name both, like with `enumerate`:

```
for name, number in contacts.items():
    print(name, "->", number)
```

```
Mum -> 98100 99999
Vikash -> 98200 22222
Riya -> 98300 33333
```

There's also `.keys()` for only the keys and `.values()` for only the values. For example, `max(counts.values())` gives the biggest value.

Pattern: counting things

This is the dictionary pattern you'll use most. How many times does each letter appear in `"banana"`?

```
counts = {}
for ch in "banana":
    counts[ch] = counts.get(ch, 0) + 1
print(counts)
print(counts["a"])
```

```
{'b': 1, 'a': 3, 'n': 2}
3
```

The middle line does all the work. Read it as: "look up how many `ch` I've counted so far (0 if this is the first one), add 1, and store it back."

```
 ch    counts.get(ch, 0)   counts after this step
 b     0                   {'b': 1}
 a     0                   {'b': 1, 'a': 1}
 n     0                   {'b': 1, 'a': 1, 'n': 1}
 a     1                   {'b': 1, 'a': 2, 'n': 1}
 n     1                   {'b': 1, 'a': 2, 'n': 2}
 a     2                   {'b': 1, 'a': 3, 'n': 2}
```

Two dictionaries are equal (`==`) when they have the same pairs, in any order. So two words have the same letters, the same number of times, exactly when their counts are equal.

```
print({"a": 1, "b": 2} == {"b": 2, "a": 1})
```

```
True
```

Pattern: have I seen this before, and where?

Here the key is a **value from the list**, and the dictionary's value is the **index** where you saw it. You walk through the list once, and at every step you can ask "have I met this number already? Where?"

Find the first number that appears twice, and both of its positions:

```python
nums = [5, 3, 8, 3, 1]
seen = {}
for i, n in enumerate(nums):
    if n in seen:
        print(f"{n} was seen before at index {seen[n]}, and again at index {i}")
        break
    seen[n] = i
print(seen)
```

```
3 was seen before at index 1, and again at index 3
{5: 0, 3: 1, 8: 2}
```

```
 i   n   n in seen?   seen afterwards
 0   5   no           {5: 0}
 1   3   no           {5: 0, 3: 1}
 2   8   no           {5: 0, 3: 1, 8: 2}
 3   3   YES, at 1    stop
```

Notice the order inside the loop: **check first, then remember**. If you stored `n` before checking, you'd always "find" the number you had stored a moment ago.

This is the idea behind **Two Sum**, the first problem in the next practice stage. There, the question at each step changes slightly: not "have I seen *this* number?" but "have I seen the number that would *complete* my answer?"

A shortcut: `Counter`

Counting is so common that Python has a ready-made counter. It lives in a toolbox called `collections`, so you **import** it first (bring it into your file):

```python
from collections import Counter

print(Counter("banana"))
```

```
Counter({'a': 3, 'n': 2, 'b': 1})
```

A `Counter` works like a dictionary, but asking for a missing key gives `0` instead of a `KeyError`. Write the counting loop yourself until it feels easy. Then use `Counter` when you want to save time. LeetCode allows the import.

Try it

Run `examples/09-dictionaries.py`:

```
python3 examples/09-dictionaries.py
```

Then change the word it counts, and add a new contact.

Common mistakes

Reading a key that isn't there.

```
counts = {}
counts["a"] += 1
```

```
KeyError: 'a'
```

`counts["a"] += 1` means `counts["a"] = counts["a"] + 1`, and the right-hand side reads `counts["a"]`, which doesn't exist yet. That's why the counting pattern uses `counts.get(ch, 0) + 1`.

Expecting `in` to search the values. `in` only looks at keys. To check values, use `x in contacts.values()`.

Check yourself

1. `ages = {"Tanvi": 20, "Vikash": 25}`. What do `ages["Tanvi"]` and `ages.get("Riya", 0)` give?
2. Count the letters of `"apple"` in your head, the way the counting pattern does. What's the final dictionary?
3. In the "seen before" pattern, what goes in the key and what goes in the value?

Next

Practise counting and looking things up: **Stage 5** in the LeetCode list (`leetcode-list.pdf`).

Example script: `examples/09-dictionaries.py`

Run it from the `learn` folder, then change things and run it again.

```python
# Page 9: Dictionaries
# Run me with:  python3 09-dictionaries.py

# key: value pairs, in curly brackets
contacts = {"Mum": "98100 11111", "Vikash": "98200 22222"}

print(contacts["Mum"])                    # 98100 11111  (look up by key)
print(contacts.get("Riya", "not saved"))  # not saved    (safe: no KeyError)

contacts["Riya"] = "98300 33333"          # new key: added
contacts["Mum"] = "98100 99999"           # old key: updated
print(len(contacts))                      # 3
print("Riya" in contacts)                 # True  (in checks keys)

for name, number in contacts.items():     # key and value together
    print(name, "->", number)

# Counting pattern
word = "mississippi"
counts = {}
for ch in word:
    counts[ch] = counts.get(ch, 0) + 1
print(counts)                   # {'m': 1, 'i': 4, 's': 4, 'p': 2}
print(max(counts.values()))     # 4  (the biggest count)

# Which letters appear more than once?
for ch, how_many in counts.items():
    if how_many > 1:
        print(ch, "appears", how_many, "times")

# A dictionary as a lookup table
price = {"tea": 10, "coffee": 30, "samosa": 15}
order = ["tea", "samosa", "samosa"]
total = 0
for item in order:
    total += price[item]
print("bill:", total)           # 40

# "Seen before, and where?" pattern: value -> index
nums = [5, 3, 8, 3, 1]
seen = {}
for i, n in enumerate(nums):
    if n in seen:               # check first...
        print(n, "first seen at", seen[n], "and again at", i)
        break
    seen[n] = i                 # ...then remember

# The shortcut
from collections import Counter
print(Counter(word))            # Counter({'i': 4, 's': 4, 'p': 2, 'm': 1})

# Try changing things:
#   1. Count the letters in your own name.
```

```
#   2. Add "chai": 12 to price and put it in the order.
#   3. Change nums so no number repeats. What does the seen-before loop print now?
```

LESSON 10

Sets and tuples

After this page you'll be able to keep a collection with no repeats, check "have I seen this?" fast, and keep two values together as a pair.

This page has two small tools. Both are simpler than lists and dictionaries, and both show up often in LeetCode answers.

Part 1: sets

A guest list

Think of the guest list at the door of a wedding. Each name is on it once: writing "Tanvi" twice doesn't make two Tanvis. Nobody cares what order the names were written in. The only question the guard asks is: **"Is this name on the list?"** And the answer has to come fast.

That's a **set**: a collection where

- every item appears **only once** (no duplicates),
- there's **no order** (no index, no "first item"),
- checking `in` is **fast**, like looking up a key in a dictionary.

Making a set

Turn a list into a set, and the repeats disappear:

```
nums = [3, 1, 3, 2, 1, 3]
unique = set(nums)
print(unique)
print(len(nums))
print(len(unique))
```

You'll see:

```
{1, 2, 3}
6
3
```

`len(set(nums))` is the quick way to count **how many different** things there are. If it's smaller than `len(nums)`, something was repeated.

A set is written with curly brackets, like a dictionary, but with single items instead of `key: value` pairs.

Adding, removing, checking

Start an empty set with `set()`, then use `add` and `remove`:

```
guests = set()
guests.add("Tanvi")
guests.add("Riya")
guests.add("Tanvi")
print(len(guests))
print("Riya" in guests)
print("Aman" in guests)
guests.remove("Riya")
print("Riya" in guests)
```

```
2
True
False
False
```

Adding `"Tanvi"` a second time changed nothing. She was already on the list.

Why use a set instead of a list for "have I seen this?" Both can answer `x in ...`. A list checks its items one by one, so a long list is slow to search. A set jumps straight to the answer, like a dictionary does. On LeetCode, with big inputs, that difference can decide whether your answer passes.

A string can become a set of its letters: `set("hello")` has 4 items (`h`, `e`, `l`, `o`), because the two `l`s count once.

Comparing two sets: `&`, `|`, `-`

```
a = {1, 2, 3, 4}
b = {3, 4, 5}
print(a & b)
print(a | b)
print(a - b)
```

```
{3, 4}
{1, 2, 3, 4, 5}
{1, 2}
```

```
a = {1, 2, 3, 4}        b = {3, 4, 5}

  only in a    in both    only in b
    1  2        3  4         5

a & b  ->  {3, 4}             in both
a | b  ->  {1, 2, 3, 4, 5}    in either
a - b  ->  {1, 2}             in a, not in b
```

- `a & b` ("and"): items in **both**.
- `a | b` ("or"): items in **either** one.
- `a - b`: items in `a` that are **not** in `b`.

If a problem wants a list back, turn the set into one with `list(...)`: `list(a & b)` gives `[3, 4]`.

Part 2: tuples

A fixed pair

Think of a map location: latitude and longitude. They always go together, and you never change one half on its own. Or a date of birth: day, month, year.

A **tuple** is a small group of values kept together that **can't be changed** once made. You write it with round brackets:

```
point = (3, 4)
print(point[0])
print(point[1])
print(len(point))
```

```
3
4
2
```

Indexing works like a list. Changing doesn't:

```
point[0] = 10
```

```
TypeError: 'tuple' object does not support item assignment
```

Same as strings. If you need it different, make a new tuple.

Unpacking: one name per value

You can take a tuple apart into separate variables in one line. This is called **unpacking**:

```
point = (3, 4)
x, y = point
print(x)
print(y)
```

```
3
4
```

The first value goes into the first name, the second into the second.

You've done this already without knowing it. `enumerate` and `.items()` hand you a tuple each time round, and `for i, n in ...` unpacks it:

```
for pair in enumerate(["a", "b"]):
    print(pair)
```

```
(0, 'a')
(1, 'b')
```

Swapping two values

Swap two variables in one line:

```
a = 1
b = 2
a, b = b, a
print(a, b)
```

```
2 1
```

Python first makes the tuple `(b, a)`, which is `(2, 1)`, and then unpacks it into `a` and `b`. You'll use this to swap two items in a list too: `nums[i], nums[j] = nums[j], nums[i]`.

Returning two values from a function

A function can only `return` one thing. But that one thing can be a tuple, so you can hand back two answers at once:

```
def smallest_and_biggest(nums):
    return min(nums), max(nums)

low, high = smallest_and_biggest([4, 9, 1, 7])
print(low)
print(high)
```

```
1
9
```

`return min(nums), max(nums)` returns the tuple `(1, 9)`, and the caller unpacks it.

Try it

Run `examples/10-sets-and-tuples.py`:

```
python3 examples/10-sets-and-tuples.py
```

Then change the lists and predict what `&`, `|` and `-` will give.

Common mistakes

`{}` is an empty dictionary, not an empty set.

```
print(type({}))
print(type(set()))
```

```
<class 'dict'>
<class 'set'>
```

For an empty set, always write `set()`.

Using an index on a set.

```
s = {1, 2, 3}
print(s[0])
```

```
TypeError: 'set' object is not subscriptable
```

A set has no order, so there's no "item 0". If you need positions, use a list.

Removing something that isn't there. `guests.remove("Aman")` when Aman isn't on the list gives `KeyError: 'Aman'`. Check with `in` first.

Unpacking into the wrong number of names. `x, y = (1, 2, 3)` gives `ValueError: too many values to unpack (expected 2)`. You need one name per value.

Check yourself

1. What is `len(set([7, 7, 7, 1]))`?
2. `a = {"x", "y"}` and `b = {"y", "z"}`. What are `a & b` and `a - b`?
3. After `p = 5`, `q = 8`, `p, q = q, p`, what are `p` and `q`?

Next

Practise with sets: **Stage 6** in the LeetCode list (`leetcode-list.pdf`).

Example script: `examples/10-sets-and-tuples.py`

Run it from the `learn` folder, then change things and run it again.

```python
# Page 10: Sets and tuples
# Run me with:  python3 10-sets-and-tuples.py

# --- Sets: no repeats, no order, fast "in" ---

nums = [3, 1, 3, 2, 1, 3]
unique = set(nums)
print(unique)                    # {1, 2, 3}
print(len(nums), len(unique))    # 6 3
print(len(unique) < len(nums))   # True: something was repeated

guests = set()                   # empty set (NOT {}, that's a dict)
guests.add("Tanvi")
guests.add("Riya")
guests.add("Tanvi")              # already there: nothing changes
print(len(guests))               # 2
print("Riya" in guests)          # True
guests.remove("Riya")
print("Riya" in guests)          # False

# "Have I seen this before?" with a set
seen = set()
for n in [4, 7, 2, 7, 9]:
    if n in seen:
        print("repeat:", n)      # repeat: 7
        break
    seen.add(n)

# Different letters in a word
print(len(set("hello")))         # 4   (h, e, l, o)

# Comparing two sets
a = {1, 2, 3, 4}
b = {3, 4, 5}
print(a & b)                     # {3, 4}           in both
print(a | b)                     # {1, 2, 3, 4, 5}  in either
print(a - b)                     # {1, 2}           in a, not in b
print(list(a & b))               # [3, 4]           back to a list

# --- Tuples: fixed groups that can't change ---

point = (3, 4)
print(point[0])                  # 3
x, y = point                     # unpacking
print(x, y)                      # 3 4

# Swap two values
left = "L"
right = "R"
left, right = right, left
print(left, right)               # R L

# Swap two items in a list
letters = ["a", "b", "c"]
```

```
letters[0], letters[2] = letters[2], letters[0]
print(letters)                  # ['c', 'b', 'a']


# Return two values at once
def smallest_and_biggest(nums):
    return min(nums), max(nums)

low, high = smallest_and_biggest([4, 9, 1, 7])
print(low, high)                # 1 9

# Try changing things:
#   1. How many different letters are in your full name? Use set and len.
#   2. Make a set of your friends and a set of Vikash's friends. Who's in both?
#   3. Write a function that returns a number's last digit AND the rest
#      (% 10 and // 10), then unpack both answers.
```